

完整软件过程定义文档

学号：2023141461061

姓名：袁新坪

项目名称：AI辅助软件过程改进平台

文档版本：V1.0

编写日期：2026年7月

1. 文档说明

1.1 目的

本文档定义“AI辅助软件过程改进平台”从项目启动、需求分析、计划、设计、开发、测试、发布、运维到复盘改进的完整软件过程。文档用于统一团队工作方式，明确各活动的输入、输出、角色、检查点、度量指标和工具支持。

1.2 适用范围

适用于：

- 课程项目开发；
- 小型团队迭代开发；
- 前后端分离系统；
- 需要Git版本管理和持续集成的项目；
- 使用AI辅助需求、代码、测试和知识管理的项目。

1.3 过程目标

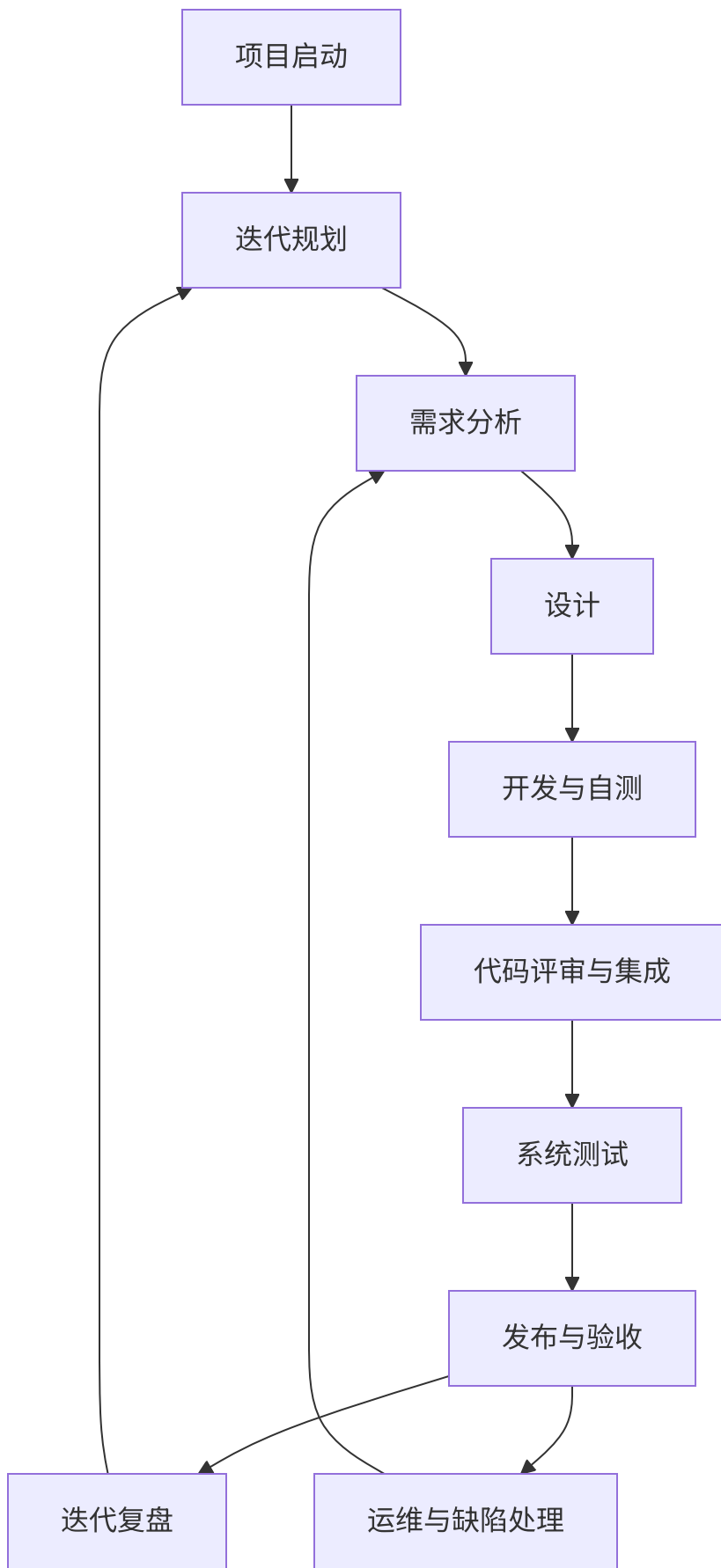
- 软件活动有明确责任人；
- 工作产品有统一模板；
- 关键活动有入口和出口标准；
- 质量问题尽早发现；
- 项目状态可追踪；
- 过程数据可度量；
- 每个迭代形成改进闭环。

2. 术语与缩写

术语	说明
需求	用户或业务对系统能力的期望
用户故事	以用户、目标和价值描述的需求形式
验收标准	判断需求是否完成的可验证条件
工作产品	过程中产生的文档、代码、测试和记录
基线	经评审确认并纳入配置管理的版本
合并请求	将分支变更合并到目标分支的申请
持续集成	提交代码后自动构建、检查和测试
质量门禁	必须满足后才能进入下一阶段的条件
缺陷	软件结果与预期要求之间的偏差
AI辅助	使用模型生成建议、草稿或分析结果
完成定义	工作项达到完成状态需要满足的条件

3. 过程模型

项目采用迭代增量模型，每个迭代完成一组可验收功能。



过程由以下过程域组成：

1. 项目管理过程；

- 2. 需求管理过程；
- 3. 技术设计过程；
- 4. 开发实现过程；
- 5. 验证与确认过程；
- 6. 配置与发布管理过程；
- 7. 质量保证过程；
- 8. 风险管理过程；
- 9. 度量分析过程；
- 10. AI使用治理过程；
- 11. 持续改进过程。

4. 生命周期状态定义

4.1 需求状态

草稿 → 待评审 → 已确认 → 开发中 → 待验收 → 已完成
 ↘ 已拒绝 / 已取消

4.2 任务状态

待处理 → 进行中 → 待评审 → 待测试 → 已完成
 ↘ 阻塞

4.3 缺陷状态

新建 → 已确认 → 修复中 → 待验证 → 已关闭
 ↘ 拒绝 / 重复 / 延后
 ↘ 重新打开

4.4 发布状态

计划中 → 准备中 → 待批准 → 发布中 → 已验证 → 已完成
 ↘ 已回滚

5. 项目启动过程

5.1 目标

明确项目目标、范围、角色、约束、主要风险和开发方式。

5.2 输入

- 课程任务要求；
- 用户或业务目标；
- 可用时间和人员；
- 技术条件；
- 已知约束。

5.3 活动

1. 编写项目章程；
2. 明确项目范围；
3. 确定团队角色；
4. 选择技术栈和工具；
5. 建立代码仓库；
6. 定义分支和提交规范；
7. 创建初始风险清单；
8. 制定里程碑计划；
9. 确认沟通方式；
10. 建立文档目录。

5.4 输出

- 项目章程；
- 角色分工表；
- 里程碑计划；
- 技术方案初稿；
- 风险登记册；
- 仓库和权限配置；
- 软件过程定义。

5.5 出口标准

- 项目目标可说明；
- 范围和不做事项明确；
- 负责人已确定；
- 仓库可使用；
- 基本计划和风险已记录。

6. 迭代规划过程

6.1 目标

选择迭代目标和工作范围，保证任务量与团队能力匹配。

6.2 活动

- 回顾上个迭代结果；
- 选择高优先级需求；
- 评估工作量；
- 识别依赖和风险；
- 拆分任务；
- 确定迭代目标；
- 确定完成定义；
- 建立测试计划。

6.3 输出

工作产品	说明
迭代目标	本迭代希望实现的结果
迭代需求列表	已选择的需求
任务清单	可执行任务及负责人
测试范围	需要验证的功能和风险
风险清单	可能影响迭代的事项

6.4 规则

- 未通过需求评审的内容不得进入开发；
- 单个任务应可在较短时间内完成；
- 任务必须有负责人；
- 阻塞任务必须记录原因；
- 临时插入需求需重新评估计划。

7. 需求管理过程

7.1 需求模板

需求标题

背景

说明为什么需要该功能。

用户故事

作为某类用户，我希望完成某项操作，以获得某种价值。

业务规则

1. ...
2. ...

前置条件

- ...

主流程

1. ...
2. ...

异常流程

1. ...
2. ...

验收标准

- Given ...
- When ...
- Then ...

非功能要求

- 性能:
- 安全:
- 可用性:

依赖与风险

- ...

7.2 需求评审角色

- 需求负责人；
- 开发代表；
- 测试代表；
- 必要时邀请项目负责人。

7.3 评审检查点

- 价值是否明确；
- 范围是否明确；
- 是否存在冲突；
- 是否可实现；
- 是否可测试；
- 是否考虑异常；
- 是否考虑权限；
- 是否考虑数据影响；
- 是否有可量化验收标准。

7.4 需求变更

变更必须记录：

- 变更原因；
- 原内容和新内容；
- 影响范围；
- 工作量变化；
- 测试影响；
- 是否影响里程碑；
- 批准人。

8. 技术设计过程

8.1 设计分类

设计类型	适用情况
简要设计	单模块、低风险变更
详细设计	跨模块、数据库或接口变更
架构决策记录	重要技术选型或长期影响决策
原型验证	技术可行性不明确

8.2 设计内容

- 目标与约束；
- 模块关系；
- 数据模型；
- 接口定义；
- 核心流程；
- 异常处理；
- 权限和安全；
- 性能考虑；
- 兼容和迁移；
- 测试策略；
- 回滚方案。

8.3 设计评审出口标准

- 主要方案可行；
- 数据影响已确认；
- 接口输入输出明确；
- 风险有应对措施；
- 测试人员可以据此设计测试；
- 重要决策已记录。

9. 开发实现过程

9.1 分支管理

分支	用途	合并要求
main	稳定版本	仅通过发布流程合并
develop	日常集成	合并请求、评审和CI通过
feature/*	功能开发	关联需求
fix/*	缺陷修复	关联缺陷
release/*	发布准备	仅允许发布相关修改

9.2 编码规则

- 遵守项目编码规范；
- 禁止硬编码密码和密钥；
- 输入必须校验；
- 异常不能无处理地吞掉；
- 日志不得输出敏感信息；
- 数据库操作考虑事务；
- 公共逻辑避免复制；
- 复杂逻辑需要注释；
- 接口变更同步更新文档；
- 新功能必须考虑测试。

9.3 开发自测

开发人员提交评审前至少完成：

- 正常流程；
- 主要异常流程；
- 边界输入；
- 权限校验；
- 数据保存和回滚；
- 接口返回；
- 日志和错误提示；

- 相关回归测试。

9.4 AI编码辅助规则

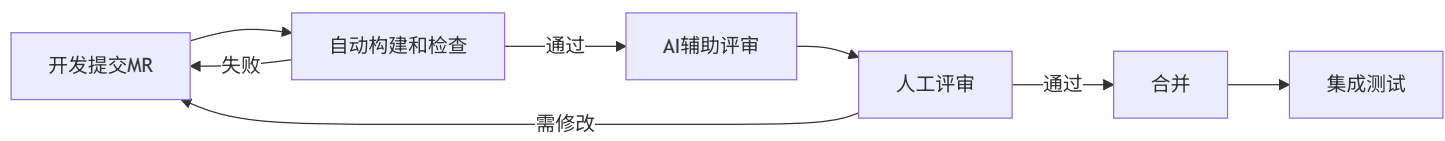
- 生成代码必须由开发人员理解；
- 不允许直接提交未经检查的代码；
- 不输入密钥、真实用户数据和敏感配置；
- 使用AI生成依赖时核对版本和许可证；
- 生成测试时检查断言有效性；
- AI生成内容应遵守项目规范。

10. 代码评审过程

10.1 合并请求内容

- 变更目的；
- 关联需求和任务；
- 主要修改；
- 测试结果；
- 风险和影响；
- 截图或接口示例；
- 数据库变更；
- 回滚说明。

10.2 评审流程



10.3 严重程度

等级	说明	处理
阻断	安全、数据丢失、严重错误	必须修复
高	主要功能错误或高风险	合并前修复

等级	说明	处理
中	可维护性或一般风险	原则上修复
低	风格或改进建议	可记录后处理

10.4 出口标准

- 自动检查通过；
- 阻断和高风险问题已解决；
- 评审意见有处理记录；
- 测试足够；
- 文档同步；
- 至少一名评审人批准。

11. 持续集成过程

11.1 流水线步骤

1. 检出代码；
2. 安装依赖；
3. 格式检查；
4. 静态分析；
5. 编译；
6. 单元测试；
7. 接口测试；
8. 安全扫描；
9. 构建制品；
10. 保存报告。

11.2 质量门禁

以下情况阻止合并：

- 编译失败；
- 单元测试失败；
- 阻断级静态分析问题；

- 发现硬编码密钥；
- 关键覆盖率低于要求；
- 依赖存在不可接受风险；
- 必要文档缺失。

12. 测试过程

12.1 测试设计

测试用例来源包括：

- 需求验收标准；
- 设计文档；
- 风险清单；
- 历史缺陷；
- 代码变更；
- AI生成建议。

12.2 测试级别

级别	负责人	目标
单元测试	开发人员	验证函数或类
接口测试	开发/测试	验证服务接口
集成测试	测试人员	验证模块协作
系统测试	测试人员	验证完整业务流程
验收测试	需求负责人	验证用户价值
回归测试	测试人员	确认已有功能未受影响

12.3 测试用例状态

- 草稿；
- 待评审；
- 已确认；

- 已执行；
- 通过；
- 失败；
- 阻塞；
- 不适用。

12.4 测试出口标准

- 计划测试已执行；
- 高优先级缺陷已关闭；
- 回归测试通过；
- 测试结果可追溯；
- 残留风险已说明；
- 验收标准全部有结论。

13. 缺陷管理过程

13.1 严重程度

等级	定义
致命	系统不可用、数据严重损坏或安全事故
严重	核心功能不可用，无合理替代方案
一般	部分功能错误，有替代方案
轻微	界面、提示或低影响问题

13.2 优先级

优先级由严重程度、用户影响、发生概率和修复成本综合确定。

13.3 根因分类

- 需求遗漏；
- 需求理解错误；
- 设计问题；

- 编码错误；
- 测试遗漏；
- 配置错误；
- 数据问题；
- 环境问题；
- 第三方依赖；
- 流程未执行。

13.4 缺陷关闭条件

- 修复代码已合并；
- 相关测试通过；
- 必要回归完成；
- 根因已分类；
- 解决方案已记录；
- 必要时增加自动化检查。

14. 配置管理过程

14.1 配置项

- 源代码；
- 需求文档；
- 设计文档；
- 数据库脚本；
- 接口文档；
- 测试用例；
- 部署脚本；
- 配置模板；
- 构建制品；
- 发布说明。

14.2 版本规则

建议采用：

主版本.次版本.修订号

- 主版本：不兼容的重要变化；
- 次版本：向后兼容的新功能；
- 修订号：向后兼容的问题修复。

14.3 基线管理

进入发布候选后，需求、代码、测试和配置形成发布基线。任何修改均需记录并重新验证。

15. 发布与部署过程

15.1 发布准备

- 确认版本范围；
- 完成测试报告；
- 生成发布说明；
- 检查配置差异；
- 检查数据库脚本；
- 制定备份和回滚；
- 确定发布时间和负责人。

15.2 发布执行

1. 备份；
2. 停止或隔离必要服务；
3. 执行数据库变更；
4. 部署应用；
5. 完成配置；
6. 启动服务；
7. 执行冒烟测试；
8. 检查日志和监控；
9. 确认发布结果。

15.3 回滚条件

- 核心功能不可用；
- 数据出现异常；
- 错误率持续超过阈值；
- 无法在规定时间内修复；
- 发现严重安全问题。

16. 运维与问题管理过程

运维活动包括：

- 服务状态检查；
- 日志收集；
- 性能监控；
- 备份验证；
- 故障处理；
- 安全更新；
- 用户反馈处理；
- 问题趋势分析。

重大故障处理后必须进行复盘，记录时间线、影响、根因、临时措施、长期措施和责任人。

17. 项目管理过程

17.1 日常跟踪

项目负责人关注：

- 迭代目标完成情况；
- 阻塞任务；
- 延期风险；
- 缺陷趋势；
- 构建状态；
- 人员负载；

- 需求变更。

17.2 沟通机制

活动	频率	目标
日常同步	每日或按需	暴露进度和阻塞
需求评审	每个需求	统一理解
技术评审	重大设计	降低技术风险
迭代评审	每个迭代	展示可交付成果
迭代复盘	每个迭代	形成改进措施

18. 风险管理过程

18.1 风险字段

- 风险编号；
- 风险描述；
- 发生概率；
- 影响程度；
- 风险等级；
- 应对措施；
- 责任人；
- 状态；
- 复查日期。

18.2 风险策略

- 避免；
- 缓解；
- 转移；
- 接受；
- 制定应急方案。

18.3 典型风险

风险	应对
需求频繁变化	设置变更评估和迭代范围保护
关键人员不可用	文档化、代码评审和知识共享
测试时间不足	测试前移、自动化和风险优先
发布失败	自动部署、备份和回滚演练
AI建议错误	人工确认和审计
数据泄露	脱敏和访问控制

19. 质量保证过程

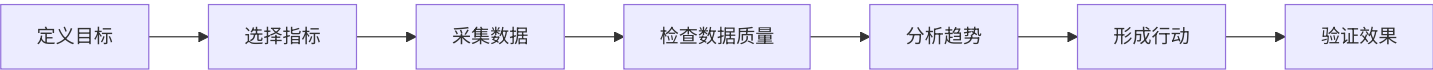
质量保证人员或指定成员定期检查：

- 需求模板执行；
- 评审记录；
- 代码评审覆盖；
- CI结果；
- 测试证据；
- 发布记录；
- 缺陷关闭条件；
- 复盘和改进措施。

发现偏差后建立纠正项，记录责任人、截止时间和验证结果。

20. 度量与分析过程

20.1 度量流程



20.2 核心指标

类别	指标
进度	交付周期、延期率、在制品数量
质量	缺陷密度、重开率、线上缺陷率
工程	构建成功率、自动测试通过率、评审等待时间
过程	模板完整率、评审覆盖率、发布检查完成率
AI	建议采纳率、误报率、时间节省率

20.3 使用原则

- 指标用于改进，不用于简单排名；
- 关注趋势，不只看单次结果；
- 数据异常先检查采集方式；
- 指标必须能够支持实际决策；
- 防止为指标而制造形式化记录。

21. AI使用治理过程

21.1 可使用场景

- 需求检查；
- 文档草稿；
- 测试场景生成；
- 代码解释；
- 常规评审建议；
- 日志摘要；
- 项目知识问答；
- 复盘材料整理。

21.2 禁止事项

- 输入密码、密钥和真实敏感数据；
- 未审核直接提交生成代码；

- 让AI自动批准发布；
- 将AI输出当作安全审计结论；
- 伪造测试结果和项目数据；
- 未经确认写入正式知识库。

21.3 输出管理

AI输出应记录：

- 使用场景；
- 输入摘要；
- 模型或工具；
- 输出结果；
- 人工评审人；
- 是否采用；
- 修改内容；
- 风险说明。

22. 持续改进过程

22.1 改进来源

- 迭代复盘；
- 缺陷根因；
- 用户反馈；
- 质量指标；
- 工具告警；
- 发布故障；
- AI使用评价；
- 流程审计。

22.2 改进步骤

1. 描述问题；
2. 收集证据；
3. 分析根因；
4. 提出候选措施；

- 5. 选择低成本试点；
- 6. 设定成功指标；
- 7. 执行；
- 8. 评价；
- 9. 标准化或撤销。

22.3 改进记录模板

字段	内容
问题	当前存在的问题
证据	数据或案例
根因	原因分析
措施	计划采取的行动
负责人	执行人员
完成时间	截止日期
成功指标	判断有效的标准
结果	实际效果
后续行动	标准化或继续调整

23. 工作产品清单

编号	工作产品	负责人
WP01	项目章程	项目负责人
WP02	需求说明	需求负责人
WP03	迭代计划	项目负责人
WP04	技术设计	开发负责人
WP05	源代码	开发人员
WP06	合并请求记录	开发与评审人员

编号	工作产品	负责人
WP07	测试计划与用例	测试人员
WP08	测试报告	测试人员
WP09	缺陷记录	全体成员
WP10	发布说明	配置管理员
WP11	风险登记册	项目负责人
WP12	度量报告	质量保证人员
WP13	复盘记录	全体成员
WP14	AI验证记录	AI管理员

24. 过程裁剪规则

对于小型低风险需求，可以裁剪详细设计，但不得裁剪：

- 需求验收标准；
- 基本测试；
- 代码评审；
- 构建检查；
- 发布记录。

对于高风险需求，应增加：

- 详细设计；
- 安全评审；
- 性能测试；
- 数据迁移验证；
- 回滚演练；
- 多人评审。

25. 过程验收标准

完整软件过程投入使用后，应满足：

- 所有主要活动有定义；
- 角色和责任明确；
- 关键工作产品有模板；
- 入口和出口条件可检查；
- 代码变更通过评审和CI；
- 测试结果可追溯到需求；
- 发布有检查和回滚；
- 缺陷有根因分类；
- 项目数据可形成指标；
- 每个迭代有改进措施；
- AI使用符合安全和审核要求。

26. 结论

本文档定义了覆盖项目全生命周期的软件过程，并将软件工程规范、自动化质量控制、项目度量和AI辅助能力统一到一个可执行框架中。该过程强调轻量化、可追溯和持续改进，适合课程项目及小型开发团队。

实际执行时，应根据项目规模进行适当裁剪，但需求验收、代码评审、自动检查、测试、发布记录和复盘等关键活动不应省略。AI工具可以提升效率，但必须接受人工审核，并纳入统一的数据安全和质量治理。

附录：完成定义

需求完成定义

- ☐ 功能已实现
- ☐ 验收标准全部通过
- ☐ 必要测试已完成
- ☐ 高优先级缺陷已关闭

- ☐ 文档已更新
- ☐ 代码已评审并合并
- ☐ CI全部通过
- ☐ 需求负责人已验收

迭代完成定义

- ☐ 迭代目标已评价
- ☐ 已完成功能可演示
- ☐ 未完成内容已重新计划
- ☐ 测试报告已形成
- ☐ 风险和缺陷已更新
- ☐ 过程指标已统计
- ☐ 复盘已完成
- ☐ 改进措施已确定